REACTJS CHUYÊN SÂU · CHƯƠNG TỔNG HỢP: GIÁO TRÌNH VÀ THỰC HÀNH · BÀI TOÀN KHÓA

ReactJS Chuyên Sâu

Tài liệu gốc kèm thực hành bổ sung · 29/10/2026

Mục lục

- React Cơ Bản: Giới thiệu React & JSX
- React Cơ Bản: Component & Props
- React Cơ Bản: State & useState
- State Management & Hooks Nâng Cao: useEffect & vòng đời component
- State Management & Hooks Nâng Cao: useContext & useReducer
- State Management & Hooks Nâng Cao: Custom Hooks
- Xây Dựng Ứng Dụng Thực Tế: Kết nối API với Axios
- Xây Dựng Ứng Dụng Thực Tế: Routing với React Router
- Xây Dựng Ứng Dụng Thực Tế: Dự án: Trang quản lý khoá học

CHƯƠNG 1 · BÀI 1

1.1. Giới thiệu React & JSX

Tài liệu học tập ReactJS Chuyên Sâu · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Hiểu React là gì và vì sao dùng React
> - Hiểu Virtual DOM và quy trình reconciliation
> - Viết JSX đúng quy tắc

1. React là gì

React là thư viện JavaScript dùng để xây dựng giao diện người dùng, do Meta (Facebook) phát triển. React chia giao diện thành các component nhỏ, độc lập và có thể tái sử dụng. Giao diện được mô tả theo kiểu khai báo: ta mô tả giao diện ứng với từng trạng thái dữ liệu, React lo việc cập nhật.

2. Virtual DOM

Virtual DOM là bản sao nhẹ của DOM thật, được lưu trong bộ nhớ dưới dạng đối tượng JavaScript. Khi state thay đổi, React so sánh Virtual DOM mới với bản cũ, quá trình này gọi là reconciliation. Sau đó React chỉ cập nhật những phần thực sự thay đổi lên DOM thật, giúp giao diện nhanh hơn.

3. JSX

JSX là cú pháp mở rộng cho phép viết mã giống HTML ngay trong JavaScript. JSX được công cụ như Babel biên dịch thành lời gọi `React.createElement`. Biểu thức JavaScript trong JSX được đặt trong dấu ngoặc nhọn, ví dụ `{user.name}`.

```
function ChaoMung() {
  const ten = "An";
  return <h1 className="tieu-de">Xin chào, {ten}!</h1>;
}
```

4. Quy tắc viết JSX

Trong JSX dùng `className` thay cho `class` và `htmlFor` thay cho `for`. Mỗi component chỉ được trả về một phần tử gốc; muốn nhóm nhiều phần tử mà không thêm thẻ thừa thì dùng Fragment `<>...</>`. Mọi thẻ trong JSX đều phải được đóng, kể cả thẻ tự đóng như `<img />`.

Tóm tắt

- React xây giao diện từ các component tái sử dụng.
- Virtual DOM và reconciliation giúp chỉ cập nhật phần thay đổi.
- JSX: `className`, `htmlFor`, một phần tử gốc (hoặc Fragment), mọi thẻ phải đóng.

Câu hỏi ôn tập và bài tập vận dụng

1. Tạo ứng dụng React bằng Vite và hiển thị lời chào có tên của bạn.
2. Viết component trả về hai thẻ liền kề bằng Fragment.
3. Giải thích vai trò của Virtual DOM.

Giới thiệu React & JSX — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: viết HTML vào JSX gây lỗi. JSX gần HTML nhưng có quy tắc JavaScript: dùng className, đóng thẻ và bọc các phần tử cùng cấp. Biểu thức đặt trong dấu ngoặc nhọn; không đặt câu lệnh if trực tiếp vào vị trí biểu thức.

```
function Greeting({name}) {
  return <section className="greeting"><h2>Chào {name}</h2></section>;
}
```

Tự luyện: render tên trống và tên dài. **Hướng giải:** chọn thông điệp thay thế có ý nghĩa; React hiển thị chuỗi như dữ liệu, không dùng dangerouslySetInnerHTML cho dữ liệu chưa tin cậy. Component cần tên viết hoa để được hiểu là component.

Nguồn học tiếp

https://react.dev/learn

CHƯƠNG 1 · BÀI 2

1.2. Component & Props

Tài liệu học tập ReactJS Chuyên Sâu · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Xây dựng function component
> - Truyền dữ liệu từ component cha xuống con bằng props
> - Hiểu props là chỉ đọc và dùng prop children

1. Component

Component là hàm JavaScript nhận dữ liệu đầu vào và trả về JSX mô tả giao diện. Tên component phải bắt đầu bằng chữ in hoa, ví dụ `UserCard`. React hiện đại dùng function component kết hợp Hooks thay cho class component.

2. Props

Props (properties) là dữ liệu mà component cha truyền xuống component con. Props chỉ đọc (read-only): component con không được tự sửa props nhận được. Muốn thay đổi dữ liệu, component cha phải thay đổi và truyền giá trị mới xuống.

```
function KhoaHocCard({ ten, gia }) {
  return (
    <div className="card">
      <h3>{ten}</h3>
      <p>{gia === 0 ? "Miễn phí" : gia + " VNĐ"}</p>
    </div>
  );
}

function App() {
  return <KhoaHocCard ten="React cơ bản" gia={0} />;
}
```

3. Prop children

Prop đặc biệt `children` chứa nội dung đặt giữa thẻ mở và thẻ đóng của component, giúp tạo các khung bao tái sử dụng.

```
function Khung({ children }) {
  return <div className="khung">{children}</div>;
}
```

4. Render danh sách

Danh sách được render bằng phương thức `map` của mảng, trả về một phần tử JSX cho mỗi mục. Mỗi phần tử trong danh sách cần thuộc tính `key` duy nhất và ổn định, thường là `id`. Không nên dùng chỉ số mảng làm key khi danh sách có thể thêm, xoá hoặc đổi thứ tự.

```
{khoaHoc.map((k) => <KhoaHocCard key={k.id} ten={k.ten} gia={k.gia} />)}
```

Tóm tắt

- Component là hàm trả về JSX, tên viết hoa chữ cái đầu.
- Props truyền từ cha xuống con và chỉ đọc; `children` là nội dung bên trong thẻ.
- Render danh sách bằng `map` kèm `key` ổn định.

Câu hỏi ôn tập và bài tập vận dụng

1. Viết component SinhVienCard nhận props tên và điểm.
2. Render danh sách 5 khoá học từ một mảng bằng map và key.
3. Giải thích vì sao không nên dùng chỉ số mảng làm key.

REACTJS CHUYÊN SÂU · CHƯƠNG 1: REACT CƠ BẢN · BÀI 2

Component & Props — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: sửa props rồi giao diện không đúng hoặc làm dữ liệu cha thay đổi. Props là đầu vào; cập nhật dữ liệu ở nơi sở hữu state và truyền callback xuống.

```
function CourseCard({course, onOpen}) {
  return <button onClick={() => onOpen(course.id)}>{course.title}</button>;
}
```

Tự luyện: render danh sách card, tìm lại một card sau khi sắp xếp. **Hướng giải:** key ổn định theo id, không theo vị trí nếu dữ liệu có đổi thứ tự. Tránh đặt nút bên trong nút; tách vùng điều hướng và hành động phụ để dùng được bằng bàn phím.

Nguồn học tiếp

https://react.dev/learn

CHƯƠNG 1 · BÀI 3

1.3. State & useState

Tài liệu học tập ReactJS Chuyên Sâu · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Hiểu state và sự khác nhau giữa state và props
> - Dùng Hook useState để quản lý trạng thái
> - Cập nhật state đúng cách và xử lý sự kiện

1. State

State là dữ liệu nội bộ của component, khi state thay đổi thì component được render lại. State do chính component quản lý, còn props được truyền từ bên ngoài vào.

2. Hook useState

Hook `useState` dùng để khai báo state trong function component. `useState` trả về một mảng gồm giá trị hiện tại và hàm cập nhật, ví dụ `const [count, setCount] = useState(0)`. Không được gán trực tiếp `count = 1`; phải gọi `setCount` để React biết cần render lại.

```
import { useState } from "react";

function BoDem() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>Đã bấm {count} lần</p>
      <button onClick={() => setCount(count + 1)}>Tăng</button>
    </div>
  );
}
```

3. Cập nhật state đúng cách

Khi giá trị mới phụ thuộc giá trị cũ, nên dùng dạng hàm: `setCount(prev => prev + 1)`. State dạng object hoặc mảng phải được thay bằng bản sao mới, ví dụ dùng toán tử spread `...`. Cập nhật state là bất đồng bộ nên giá trị chưa đổi ngay sau lời gọi hàm set.

```
const [ds, setDs] = useState([]);
setDs([...ds, "Mục mới"]);          // thêm
setDs(ds.filter((x) => x !== "a")); // xoá
```

4. Xử lý sự kiện và form

Sự kiện trong React viết theo kiểu camelCase, ví dụ `onClick` và `onChange`. Hàm xử lý được truyền dưới dạng tham chiếu: `onClick={handleClick}`, không phải `onClick={handleClick()}`. Để chặn hành vi mặc định của form, gọi `e.preventDefault()` trong hàm xử lý `onSubmit`. Ô nhập liệu điều khiển bằng state (controlled input) có `value` lấy từ state và `onChange` cập nhật state.

Tóm tắt

- State là dữ liệu nội bộ; thay đổi state làm component render lại.
- `useState` trả về cặp `[giá trị, hàm set]`; luôn dùng hàm set, không gán trực tiếp.
- State mảng hoặc object phải thay bằng bản sao mới; dùng dạng hàm khi phụ thuộc giá trị cũ.

Câu hỏi ôn tập và bài tập vận dụng

1. Làm bộ đếm có ba nút Tăng, Giảm và Đặt lại.
2. Làm form nhập tên hiển thị lời chào ngay khi gõ (controlled input).
3. Làm danh sách công việc cho phép thêm và xoá mục.

REACTJS CHUYÊN SÂU · CHƯƠNG 1: REACT CƠ BẢN · BÀI 3

State & useState — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: gọi setCount nhiều lần nhưng tăng không như dự đoán. Khi state mới phụ thuộc state trước, dùng updater. Không sửa trực tiếp mảng/object rồi truyền lại cùng tham chiếu.

```
setCount(previous => previous + 1);
setCourses(previous => previous.map(c => c.id === id ? {...c, title} : c));
```

Tự luyện: đếm hai lần tăng trong một handler và sửa tên một khóa. **Hướng giải:** coi state như snapshot của lần render; console.log ngay sau setState có thể vẫn thấy snapshot cũ. Trạng thái suy ra từ dữ liệu, như số mục đã hoàn thành, thường có thể tính khi render thay vì lưu thêm state dễ lệch nhau.

Nguồn học tiếp

https://react.dev/learn

CHƯƠNG 2 · BÀI 1

2.1. useEffect & vòng đời component

Tài liệu học tập ReactJS Chuyên Sâu · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Hiểu side effect và dùng Hook useEffect
> - Điều khiển thời điểm chạy effect bằng mảng phụ thuộc
> - Dùng hàm cleanup và hiểu vòng đời component

1. Side effect

Side effect là những việc ngoài việc vẽ giao diện như gọi API, đăng ký sự kiện hoặc hẹn giờ. `useEffect` là Hook dùng để chạy side effect. `useEffect` nhận hai tham số: hàm effect và mảng phụ thuộc (dependency array).

2. Mảng phụ thuộc

Mảng phụ thuộc quyết định khi nào effect chạy lại.

- `useEffect` với mảng rỗng `[]` chỉ chạy một lần sau lần render đầu tiên, khi component được gắn vào (mount).
- `useEffect` có mảng `[id]` chạy lại mỗi khi giá trị `id` thay đổi.
- `useEffect` không có mảng phụ thuộc chạy lại sau mọi lần render.

```
useEffect(() => {
  document.title = `Bạn đã bấm ${count} lần`;
}, [count]);
```

3. Hàm cleanup

Hàm được return trong effect gọi là cleanup, chạy trước khi effect chạy lại hoặc khi component bị gỡ (unmount). Cleanup dùng để huỷ hẹn giờ, huỷ đăng ký sự kiện hoặc huỷ yêu cầu API đang chờ.

```
useEffect(() => {
  const t = setInterval(() => setGiay((g) => g + 1), 1000);
```

```
  return () => clearInterval(t);   // dọn dẹp khi gỡ component
}, []);
```

4. Vòng đời component

Vòng đời component gồm ba giai đoạn: mount (gắn vào), update (cập nhật) và unmount (gỡ bỏ). Với function component, cả ba giai đoạn đều được xử lý qua `useEffect`. Quên cleanup khi unmount có thể gây rò rỉ bộ nhớ (memory leak).

Tóm tắt

- `useEffect` chạy side effect sau khi render.
- `[]` chạy một lần khi mount; `[dep]` chạy lại khi dep đổi; không mảng thì chạy sau mọi lần render.
- Hàm return trong effect là cleanup, dùng để dọn dẹp.

Câu hỏi ôn tập và bài tập vận dụng

1. Viết component đồng hồ đếm giây, có dọn dẹp khi gỡ component.
2. Dùng useEffect đổi tiêu đề trang theo số lần bấm nút.
3. Giải thích điều gì xảy ra nếu quên mảng phụ thuộc khi gọi API trong useEffect.

useEffect & vòng đời component — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: đổi khóa nhanh làm response cũ ghi đè response mới. Effect đồng bộ với hệ thống bên ngoài, cần cleanup và dependency đúng; không dùng effect để thay thế mọi phép tính.

```
useEffect(() => {
  const controller = new AbortController();
  loadCourses(controller.signal);
  return () => controller.abort();
}, [courseId]);
```

Tự luyện: làm chậm mạng và chuyển giữa hai khóa. **Hướng giải:** bỏ qua response đã hết hiệu lực; xử lý AbortError phù hợp. StrictMode trong phát triển có thể chạy thêm chu kỳ setup/cleanup để phát hiện lỗi; không tắt nó chỉ để che request/side effect bị viết sai. Không để dependency rỗng khi effect thực sự phụ thuộc courseId.

Nguồn học tiếp

https://react.dev/learn

CHƯƠNG 2 · BÀI 2

2.2. useContext & useReducer

Tài liệu học tập ReactJS Chuyên Sâu · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Dùng Context để tránh prop drilling
> - Dùng Hook useReducer quản lý state phức tạp
> - Kết hợp Context với Reducer để quản lý state toàn ứng dụng

1. Prop drilling và Context

Context là cơ chế truyền dữ liệu xuống nhiều tầng component mà không cần truyền props qua từng tầng. Tình trạng phải truyền props qua nhiều tầng trung gian được gọi là prop drilling. Context được tạo bằng `createContext`, cung cấp bằng Provider và đọc bằng Hook `useContext`.

```
const ThemeContext = createContext("sang");

function App() {
  return (
    <ThemeContext.Provider value="toi">
      <NutBam />
    </ThemeContext.Provider>
  );
}

function NutBam() {
  const theme = useContext(ThemeContext);
  return <button className={theme}>Bấm</button>;
}
```

2. useReducer

`useReducer` là Hook quản lý state phức tạp bằng một hàm reducer. Reducer là hàm thuần nhận state hiện tại và action, rồi trả về state mới. Component gửi action bằng hàm `dispatch`, ví dụ `dispatch({ type: 'add', payload: item })`.

```
function reducer(state, action) {
  switch (action.type) {
    case "add":    return [...state, action.payload];
    case "remove": return state.filter((x) => x.id !== action.id);
    default:       return state;
  }
}
const [gioHang, dispatch] = useReducer(reducer, []);
```

3. Khi nào dùng

Dùng `useState` cho state đơn giản. Dùng `useReducer` khi state có nhiều trường hoặc nhiều cách cập nhật liên quan nhau. Dùng Context khi nhiều component ở các tầng khác nhau cần cùng dữ liệu như người dùng đăng nhập, giao diện sáng hoặc tối.

4. Lưu ý hiệu năng

Mỗi khi giá trị của Provider thay đổi, mọi component dùng context đó đều render lại. Nên tách context theo chức năng (đăng nhập, giao diện) thay vì gom mọi thứ vào một context lớn.

Tóm tắt

- Context tránh prop drilling: `createContext`, Provider, `useContext`.
- `useReducer` quản lý state phức tạp bằng reducer thuần và `dispatch` action.
- Tách context theo chức năng để hạn chế render lại không cần thiết.

Câu hỏi ôn tập và bài tập vận dụng

1. Tạo ThemeContext cho phép đổi giao diện sáng và tối toàn ứng dụng.
2. Viết giỏ hàng bằng useReducer với các action thêm, xoá, đổi số lượng.
3. Giải thích khi nào nên chọn useReducer thay cho useState.

useContext & useReducer — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: nhiều trang cần user hiện tại hoặc giỏ thao tác. Context truyền giá trị, reducer mô tả cách chuyển trạng thái; không thay thế mọi state cục bộ bằng context toàn ứng dụng.

```
function reducer(state, action) {
  switch (action.type) {
    case "reset": return {items: []};
    case "add": return {...state, items: [...state.items, action.item]};
    default: return state;
  }
}
```

Tự luyện: thêm rồi reset, kiểm tra state đầu không bị sửa. **Hướng giải:** reducer không gửi HTTP hoặc tạo side effect; auth context không phải lớp bảo mật thay cho API. Backend vẫn kiểm tra token và quyền ở mọi request.

Nguồn học tiếp

https://react.dev/learn

CHƯƠNG 2 · BÀI 3

2.3. Custom Hooks

Tài liệu học tập ReactJS Chuyên Sâu · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Hiểu custom Hook và lợi ích của nó
> - Viết custom Hook để tái sử dụng logic có state
> - Nắm các quy tắc của Hooks

1. Custom Hook là gì

Custom Hook là hàm JavaScript tự viết, có tên bắt đầu bằng `use` và gọi các Hook khác bên trong. Custom Hook dùng để tái sử dụng logic có state giữa nhiều component, ví dụ `useFetch` hoặc `useLocalStorage`, mà không cần lặp lại code.

2. Ví dụ useLocalStorage

```
import { useState, useEffect } from "react";

function useLocalStorage(khoa, giaTriMacDinh) {
  const [giaTri, setGiaTri] = useState(() => {
    const luu = localStorage.getItem(khoa);
    return luu ? JSON.parse(luu) : giaTriMacDinh;
  });

  useEffect(() => {
    localStorage.setItem(khoa, JSON.stringify(giaTri));
  }, [khoa, giaTri]);

  return [giaTri, setGiaTri];
}

// Dùng: const [theme, setTheme] = useLocalStorage("theme", "sang");
```

3. Ví dụ useFetch

Custom Hook gọi API trả về ba trạng thái dữ liệu, đang tải và lỗi, để mọi component dùng chung.

```
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let huy = false;
    fetch(url)
      .then((r) => r.json())
      .then((d) => !huy && setData(d))
      .catch((e) => !huy && setError(e))
      .finally(() => !huy && setLoading(false));
    return () => { huy = true; };
  }, [url]);

  return { data, loading, error };
}
```

4. Quy tắc của Hooks

Chỉ gọi Hook ở cấp cao nhất của component, không gọi trong vòng lặp, điều kiện hay hàm lồng nhau. Chỉ gọi Hook trong function component hoặc trong custom Hook. Mỗi lần gọi custom Hook tạo ra state riêng, các component dùng chung hook không dùng chung state.

Tóm tắt

- Custom Hook là hàm bắt đầu bằng `use`, dùng để tái sử dụng logic có state.
- Ví dụ thường gặp: `useLocalStorage`, `useFetch`.
- Tuân thủ quy tắc Hooks: chỉ gọi ở cấp cao nhất, trong component hoặc custom Hook.

Câu hỏi ôn tập và bài tập vận dụng

1. Viết custom Hook `useToggle` trả về giá trị bật tắt và hàm đảo giá trị.
2. Dùng `useFetch` hiển thị danh sách khoá học từ API.
3. Giải thích vì sao không được gọi Hook bên trong câu lệnh if.

Custom Hooks — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: ba trang lặp logic loading/error và hủy request. Custom Hook tái sử dụng logic stateful, không tự làm các component dùng chung cùng một state nếu không thiết kế như vậy.

Tự luyện: thiết kế `useCourse(id)` trả data, loading, error và reload; viết trước hợp đồng khi id rỗng và khi request thất bại. **Hướng giải:** gọi Hooks ở cấp cao nhất của component/Hook, không gọi có điều kiện hoặc trong vòng lặp. Hook nên trả lỗi để UI quyết định thông báo; không nuốt lỗi thành dữ liệu rỗng vì người dùng sẽ hiểu nhầm "không có khóa".

Nguồn học tiếp

https://react.dev/learn

CHƯƠNG 3 · BÀI 1

3.1. Kết nối API với Axios

Tài liệu học tập ReactJS Chuyên Sâu · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Gọi API từ ứng dụng React bằng Axios
> - Quản lý ba trạng thái: đang tải, thành công, lỗi
> - Cấu hình Axios dùng chung và gắn token xác thực

1. Axios là gì

Axios là thư viện gửi yêu cầu HTTP dựa trên Promise, dùng được trong trình duyệt và Node.js. So với `fetch`, Axios tự chuyển dữ liệu JSON, tự báo lỗi với mã trạng thái 4xx, 5xx và hỗ trợ interceptor.

2. Gọi API trong component

Lời gọi API thường đặt trong `useEffect` và dùng `async/await` để chờ kết quả. Nên quản lý ba trạng thái khi tải dữ liệu: đang tải (loading), thành công (data) và lỗi (error).

```
import axios from "axios";
import { useEffect, useState } from "react";

function DanhSachKhoaHoc() {
  const [ds, setDs] = useState([]);
  const [loading, setLoading] = useState(true);
  const [loi, setLoi] = useState(null);

  useEffect(() => {
    (async () => {
      try {
        const res = await axios.get("/api/courses");
        setDs(res.data.data);
      } catch (e) {
        setLoi("Không tải được dữ liệu");
      } finally {
        setLoading(false);
      }
```

```
    })();
  }, []);

  if (loading) return <p>Đang tải...</p>;
  if (loi) return <p>{loi}</p>;
  return <ul>{ds.map((k) => <li key={k.id}>{k.title}</li>)}</ul>;
}
```

3. Axios instance và interceptor

Tạo một instance dùng chung với địa chỉ gốc và gắn token vào mọi yêu cầu bằng interceptor.

```
const api = axios.create({ baseURL: "http://127.0.0.1:8080" });
api.interceptors.request.use((cfg) => {
  const token = localStorage.getItem("token");
  if (token) cfg.headers.Authorization = `Bearer ${token}`;
  return cfg;
});
```

4. Gửi dữ liệu

Dùng `api.post(url, dulieu)` để tạo mới, `api.put` để cập nhật, `api.delete` để xoá. Sau khi gửi thành công nên cập nhật lại state để giao diện phản ánh dữ liệu mới.

Tóm tắt

- Axios gửi yêu cầu HTTP bằng Promise, tự xử lý JSON.
- Quản lý ba trạng thái loading, data, error khi tải dữ liệu.
- Dùng axios instance và interceptor để cấu hình chung và gắn token.

Câu hỏi ôn tập và bài tập vận dụng

1. Hiển thị danh sách khoá học từ API với trạng thái đang tải và lỗi.
2. Tạo axios instance gắn token đăng nhập vào mọi yêu cầu.
3. Viết form thêm khoá học gọi API POST rồi làm mới danh sách.

Kết nối API với Axios — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: server trả 401, 403 và 500 nhưng UI đều hiện danh sách rỗng. Phân biệt lỗi xác thực, quyền, dữ liệu đầu vào và lỗi hệ thống; không hiển thị token hoặc stack trace cho người học.

Tự luyện: mô phỏng 200 có dữ liệu, 200 rỗng, 401, 403, 422 và mất kết nối. **Hướng giải:** mỗi trạng thái có thông báo/hành động phù hợp. Khi gửi mutation, chống click lặp bằng trạng thái chờ và kiểm soát trùng phía backend. Nếu dự án bọc Axios thành `{ok, data}`, hãy kiểm tra hợp đồng wrapper thay vì dùng lẫn cú pháp response Axios nguyên bản.

Nguồn học tiếp

https://react.dev/learn

CHƯƠNG 3 · BÀI 2

3.2. Routing với React Router

Tài liệu học tập ReactJS Chuyên Sâu · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Điều hướng nhiều trang trong ứng dụng React bằng React Router
> - Khai báo route, dùng Link, useParams và useNavigate
> - Bảo vệ trang yêu cầu đăng nhập

1. React Router là gì

React Router là thư viện điều hướng giữa các trang trong ứng dụng một trang (SPA). Các route được khai báo bằng thành phần `Routes` và `Route`, mỗi `Route` gắn `path` với một `element`. Thành phần `Link` dùng để chuyển trang mà không tải lại toàn bộ trang web.

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <nav><Link to="/">Trang chủ</Link> <Link to="/khoa-hoc">Khoá học</Link></nav>
      <Routes>
        <Route path="/" element={<TrangChu />} />
        <Route path="/khoa-hoc" element={<DanhSachKhoaHoc />} />
        <Route path="/khoa-hoc/:id" element={<ChiTietKhoaHoc />} />
        <Route path="*" element={<p>Không tìm thấy trang</p>} />
      </Routes>
    </BrowserRouter>
  );
}
```

2. useParams và useNavigate

Hook `useParams` dùng để đọc tham số trên URL, ví dụ `id` trong đường dẫn `/khoa-hoc/:id`. Hook `useNavigate` dùng để chuyển trang bằng code, ví dụ sau khi đăng nhập thành công. Hook `useSearchParams` đọc tham số sau dấu hỏi như `?page=2`.

```
function ChiTietKhoaHoc() {
  const { id } = useParams();
  const navigate = useNavigate();
  return <button onClick={() => navigate("/khoa-hoc")}>Quay lại (đang xem {id})
</button>;
}
```

3. Route được bảo vệ

Để chặn người chưa đăng nhập, bọc route trong một component kiểm tra đăng nhập; nếu chưa đăng nhập thì chuyển hướng bằng `<Navigate to="/login" replace />`.

```
function RequireAuth({ children }) {
  const dangNhap = Boolean(localStorage.getItem("token"));
  return dangNhap ? children : <Navigate to="/login" replace />;
}
```

4. Route lồng nhau

Các route có chung bố cục (thanh menu, thanh bên) có thể lồng nhau; component cha hiển thị phần chung và dùng `<Outlet />` để chừa chỗ cho route con.

Tóm tắt

- `Routes`/`Route` khai báo đường dẫn; `Link` chuyển trang không tải lại.
- `useParams` đọc tham số URL, `useNavigate` chuyển trang bằng code.
- Bảo vệ route bằng component kiểm tra đăng nhập và `Navigate`.

Câu hỏi ôn tập và bài tập vận dụng

1. Tạo ứng dụng có ba trang: trang chủ, danh sách khoá học, chi tiết khoá học.
2. Thêm trang đăng nhập và chặn trang khoá học khi chưa đăng nhập.
3. Dùng route lồng nhau để dùng chung thanh menu cho nhiều trang.

Routing với React Router — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: copy URL trang kết quả sang tab mới nhưng thiếu dữ liệu. Navigation state chỉ là tiện ích trong phiên điều hướng; nội dung cần mở lại phải có id trong URL và tải lại được từ API có xác thực.

```
<Link to={`/courses/${course.id}`}>{course.title}</Link>
```

Tự luyện: điều hướng, F5, nút Back và URL không tồn tại. **Hướng giải:** dùng API của phiên bản React Router đang cài; guard phía client hỗ trợ UX, backend mới quyết định quyền. Trang 404 và thiếu tham số cần thông báo rõ, không âm thầm hiển thị dữ liệu khóa mặc định khác.

Nguồn học tiếp

https://react.dev/learn

CHƯƠNG 3 · BÀI 3

3.3. Dự án: Trang quản lý khoá học

Tài liệu học tập ReactJS Chuyên Sâu · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Xây dựng ứng dụng quản lý khoá học hoàn chỉnh bằng React
> - Kết hợp component, state, hooks, Axios và React Router
> - Tổ chức thư mục dự án và tách logic thành custom Hook

1. Yêu cầu

Ứng dụng gồm: trang đăng nhập, trang danh sách khoá học có tìm kiếm, trang chi tiết khoá học, form thêm và sửa khoá học, và chức năng xoá có xác nhận. Dữ liệu được lấy từ API của backend.

2. Tổ chức thư mục

- `src/api/`: cấu hình Axios và các hàm gọi API (`courseApi.js`).
- `src/components/`: các component dùng lại (`CourseCard`, `Loading`, `Modal`).
- `src/pages/`: từng trang (`Login`, `CourseList`, `CourseDetail`, `CourseForm`).
- `src/hooks/`: custom Hook (`useAuth`, `useCourses`).
- `src/context/`: `AuthContext` lưu thông tin đăng nhập.

3. Luồng chính

1. `AuthContext` giữ người dùng và token; `RequireAuth` bảo vệ các trang.
2. `CourseList` dùng `useCourses` để tải danh sách, lọc theo từ khoá nhập trong ô tìm kiếm.
3. `CourseForm` dùng controlled input, kiểm tra dữ liệu rồi gọi API tạo hoặc cập nhật.
4. Sau khi thêm, sửa hoặc xoá thành công, cập nhật lại state để danh sách đổi ngay.

```
function useCourses() {
  const [courses, setCourses] = useState([]);
  const [loading, setLoading] = useState(true);
  const tai = async () => {
    setLoading(true);
    const res = await api.get("/api/courses");
    setCourses(res.data.data);
```

```
    setLoading(false);
  };
  useEffect(() => { tai(); }, []);
  return { courses, loading, tai };
}
```

4. Gợi ý mở rộng

Phân trang và sắp xếp danh sách; thông báo (toast) khi thao tác thành công hoặc lỗi; chế độ giao diện sáng tối lưu bằng localStorage; kiểm thử component bằng React Testing Library.

Tóm tắt

- Tách thư mục theo trách nhiệm: api, components, pages, hooks, context.
- Dùng Context cho đăng nhập, custom Hook cho logic tải dữ liệu, Router cho điều hướng.
- Cập nhật state sau mỗi thao tác để giao diện luôn đúng dữ liệu.

Câu hỏi ôn tập và bài tập vận dụng

1. Hoàn thiện ứng dụng với đầy đủ thêm, sửa, xoá và tìm kiếm khoá học.
2. Thêm phân trang cho danh sách khoá học.
3. Thêm thông báo khi thao tác thành công hoặc thất bại.

CHƯƠNG HỌC THÊM · BÀI TỔNG HỢP

Học thêm.Tổng hợp. Học thêm, bài tập và hướng dẫn giải

Tài liệu học tập ReactJS Chuyên Sâu · Nội dung biên soạn trong dự án StudyOnline.

React — Học thêm về state, effect và ứng dụng quản lý khóa học

Ví dụ hướng đến React 18 đang dùng trong StudyOnline. Tài liệu bổ sung cho JSX, props, state, hooks, API và routing.

Mục tiêu học thêm

- Cập nhật state đúng khi phụ thuộc trạng thái trước
- Phân biệt dữ liệu suy ra, sự kiện và effect
- Xử lý request cũ và thiết kế trạng thái tải/lỗi/rỗng

1. State là ảnh chụp của lần render

Gọi setter yêu cầu lần render mới; biến state trong hàm xử lý hiện tại không lập tức biến thành giá trị mới. Khi cập nhật phụ thuộc giá trị trước, dùng updater function.

```
function Counter() {
  const [count, setCount] = useState(0);
  function addTwo() {
    setCount(c => c + 1);
    setCount(c => c + 1);
  }
  return <button onClick={addTwo}>{count}</button>;
}
```

Ví dụ cần import useState từ react. Hai updater được áp dụng liên tiếp nên tăng 2. Tránh thay đổi trực tiếp mảng/đối tượng trong state; tạo giá trị mới để React và các component con nhận ra thay đổi.

```
setCourses(previous => previous.map(course =>
  course.id === editedId ? { ...course, title: newTitle } : course
));
```

key khi render danh sách nên là mã ổn định. Dùng chỉ số khi thêm/xóa/sắp xếp có thể làm state của một hàng gắn nhầm sang hàng khác.

2. Khi nào cần effect?

Effect dùng đồng bộ với hệ thống bên ngoài như kết nối, timer hay request. Nếu chỉ lọc danh sách từ state hiện có, tính trong render thay vì lưu thêm state rồi cập nhật bằng effect.

```
const visibleCourses = courses.filter(course =>
  course.title.toLowerCase().includes(search.toLowerCase())
);
```

Thao tác lưu do người dùng bấm nút thường thuộc event handler. Effect không nên được dùng để suy đoán rằng người dùng vừa bấm nút chỉ vì state thay đổi. Với dependency, liệt kê các giá trị reactive thật sự được effect đọc; bỏ dependency tùy tiện dễ làm effect dùng dữ liệu cũ.

3. Request và cleanup

```
useEffect(() => {
  const controller = new AbortController();
  let active = true;
  setLoading(true);
  setError(null);
  fetch('/api/courses', { signal: controller.signal })
    .then(response => {
      if (!response.ok) throw new Error('Không tải được khóa học');
      return response.json();
    })
    .then(payload => { if (active) setCourses(payload.data || []); })
    .catch(error => {
      if (active && error.name !== 'AbortError') setError(error.message);
    })
    .finally(() => { if (active) setLoading(false); });
  return () => { active = false; controller.abort(); };
}, []);
```

Đây là phần đặt trong component có các state tương ứng; schema payload phải đối chiếu API thật. Cleanup ngăn request cũ cập nhật component không còn cần dữ liệu. Trong Strict Mode phát triển, effect có thể được chạy setup/cleanup thêm để phát hiện lỗi; cần cleanup đúng thay vì tắt Strict Mode để che lỗi.

4. Context, reducer và routing

Context chia sẻ dữ liệu như người dùng đăng nhập; không tự thay thế mọi state cục bộ. Reducer hữu ích khi nhiều thao tác phải cập nhật các phần liên quan theo quy tắc rõ ràng. Reducer nên thuần, không gọi API bên trong.

Frontend chặn route theo vai trò giúp trải nghiệm rõ ràng nhưng backend vẫn phải kiểm tra quyền mỗi request. Không xem việc ẩn nút sửa khóa là bảo vệ dữ liệu. Khi route đổi id khóa học, effect tải dữ liệu cần phụ thuộc id và hủy dữ liệu/request cũ phù hợp.

Câu hỏi ôn tập và bài tập vận dụng

1. Dễ: giải thích hai lần setCount(count + 1) khác hai updater như thế nào. Khi cùng đọc ảnh chụp count, cả hai có thể yêu cầu cùng một giá trị tăng 1.
2. Trung bình: lọc danh sách theo từ khóa mà không thêm state visibleCourses. Tính bằng filter; giữ courses là danh sách gốc.
3. Khó: chuyển nhanh giữa hai trang chi tiết khóa khi mạng chậm. Chứng minh dữ liệu khóa cũ không ghi đè khóa mới; thử bằng làm chậm network và cleanup request.

Dự án mở rộng và tiêu chí

Làm trang quản lý khóa với xem/thêm/sửa/xóa, xác nhận thao tác xóa và trạng thái lỗi. Chỉ cập nhật thành công khi API thành công, hoặc triển khai cập nhật lạc quan kèm hoàn tác rõ ràng. Kiểm thử danh sách rỗng, lỗi 401, lỗi 403, gửi form hai lần và mất mạng. Component giao diện cần thể hiện tải/lỗi/rỗng/thành công; kiểm tra quyền cuối cùng thuộc backend.

Nguồn học tiếp

https://react.dev/learn/synchronizing-with-effects

https://react.dev/learn/queueing-a-series-of-state-updates

https://react.dev/learn/updating-arrays-in-state

Dự án: Trang quản lý khoá học — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Bài thực hành: danh sách có tìm kiếm, lọc trạng thái, chi tiết và form tạo/sửa trên API lab. Phân biệt trạng thái "chưa làm", "đang làm", "đã nộp" nếu có quiz; điểm đạt là thông tin phụ, không đồng nghĩa đã nộp và đạt là cùng một trạng thái.

Tự luyện: tìm không có kết quả, gửi lỗi, thao tác nhanh, mở trên điện thoại và đăng xuất rồi Back.
Hướng giải: sau mutation chỉ cập nhật UI khi server xác nhận hoặc có cơ chế rollback rõ; kiểm tra quyền trước và sau khi tải lại. Báo cáo cần ghi những thao tác đã thử, không suy ra toàn bộ backend an toàn chỉ từ việc menu bị ẩn.

Nguồn học tiếp

https://react.dev/learn